

Implementación y Análisis de Complejidad

Clelia Edithzoe Alzate León

Docente: David Alberto Herrera Alvarez

30 de septiembre de 2026

Universidad Nacional de Colombia

1. OBJETIVO

Analizar y comparar el comportamiento de las estructuras de datos *List*, *Stack* y *Queue*, así como sus implementaciones mediante *DinamicArray* y *LinkedList* en Java, con el fin de identificar las condiciones más adecuadas para su uso, evaluando ventajas, desventajas y aplicaciones en escenarios reales.

2. IMPLEMENTACIÓN

2.1. Implementación de *LinkedList*

2.1.1. *Nodo*: Se implementó una clase genérica *Node* encargada de almacenar el dato (*data*) y una referencia al siguiente nodo (*next*). En el caso de la *DoublyLinkedList* también se incluyó una referencia al nodo anterior (*prev*).

```
private static class Node {  
    int data;  
    Node next;  
  
    Node(int data) {  
        this.data = data;  
    }  
}
```

El constructor *Node()* recibe el valor que se desea almacenar y lo asigna al atributo *data* mediante la referencia *this*, que permite distinguir el atributo de la clase del parámetro recibido.

2.1.2. *SinglyLinkedListNoTail*: En la implementación de la *SinglyLinkedListNoTail*, se crearon dos atributos; *head* que mantiene la referencia al primer nodo de la estructura y *size* que almacena la cantidad actual de elementos, permitiendo conocer el tamaño de la lista sin necesidad de recorrerla completamente.

```
private Node head;  
private int size;
```

Métodos

- *PushBack* ($O(n)$): El método *pushBack* inserta un nuevo nodo al *final* de la lista.

```
actual.next = newNode;  
size++;
```

- *PopFront* ($O(1)$): El método *popFront* elimina y devuelve el valor del *primer* nodo de la lista.

```

    head = head.next;
    size--;
    return item;

```

- *PopBack* ($O(n)$): El método *popBack* elimina y devuelve el valor del *último* nodo de la lista.

```

    actual.next = null;
    size--;
    return item;

```

- *Erase* ($O(n)$): El método *erase* elimina el primer nodo que contiene el valor especificado.

```

public boolean erase(int item) {
    if (head == null)
        return false;
    if (head.data == item) {
        head = head.next;
        size--;
        return true;
    }
}

```

- *AddAfter* ($O(1)$): El método *addAfter* añade un nuevo nodo con el valor *newItem* después del primer nodo que contiene el valor *target*.

```

public void addAfter(int target, int newItem) {
    if (head == null)
        return;
    Node actual = head;
    while (actual != null) {
        if (actual.data == target) {
            Node newNode = new Node(newItem);
            newNode.next = actual.next;
            actual.next = newNode;
            size++;
            return;
        }
        actual = actual.next;
    }
}

```

- *Back* ($O(n)$): El método *back* devuelve el valor del último nodo de la lista.

```

while (actual.next != null)
    actual = actual.next;
return actual.data;

```

2.1.3. *SinglyLinkedListWithTail*: En la implementación de la *SinglyLinkedListWithTail*, se crearon tres atributos; *head* que mantiene la referencia al primer nodo de la estructura, *tail* que mantiene la referencia al último nodo de la lista y *size* que almacena la cantidad actual de elementos, permitiendo conocer el tamaño de la lista sin necesidad de recorrerla completamente.

```

private Node head;
private Node tail;
private int size;

```

Métodos

- *PushBack* ($O(1)$): El método *pushBack* inserta un nuevo nodo al final de la lista.

```
tail.next = newNode;
tail = newNode;
size++;
```

- *PopFront* ($O(1)$): El método *popFront* elimina y retorna el valor almacenado en el primer nodo de la lista.

```
public int popFront() {
    if (head == null) {
        return -1;
    }
    int item = head.data;
    head = head.next;
    if (head == null) {
        tail = null;
    }
    size--;
    return item;
}
```

- *PopBack* ($O(n)$): El método *popBack* elimina y devuelve el valor del *último* nodo de la lista.

```
int item = actual.next.data;
actual.next = null;
tail = actual;
size--;
return item;
```

- *Erase* ($O(n)$): El método *erase* elimina el primer nodo con el valor especificado.

```
if (head.data == item) {
    head = head.next;
    if (head == null) {
        tail = null;
    }
    size--;
    return true;
}
```

- *AddAfter* ($O(1)$): El método *addAfter* inserta un nuevo nodo con el valor *newItem* después del primer nodo que contiene el valor *target*.

```
while (actual != null) {
    if (actual.data == target) {
        Node newNode = new Node(newItem);
        newNode.next = actual.next;
        actual.next = newNode;
    }
}
```

- *Back* ($O(1)$): El método *back* retorna el valor almacenado en el último nodo de la lista mediante la referencia *tail*.

```
public int back() {
    if (tail == null) {
        return -1;
    }
    return tail.data;
}
```

Métodos que no cambian *WithTail* y *NoTail* en *SinglyLinkedList*

- *AddBefore* ($O(n)$): El método *addBefore* inserta un nuevo nodo con el valor *newItem* antes del primer nodo que contiene el valor *target*.

```

while (actual != null) {
    if (actual.data == target) {
        Node newNode = new Node(newItem);
        newNode.next = actual;
        prev.next = newNode;
        size++;
        return;
    }
    prev = actual;
    actual = actual.next;
}

```

- *PushFront* ($O(1)$): El método *pushFront* inserta un nuevo nodo al inicio de la lista.

```

public void pushFront(int item) {
    Node newNode = new Node(item);
    newNode.next = head;
    head = newNode;
    size++;
}

```

2.1.4. *DoublyLinkedListNoTail*: En la implementación de la *DoublyLinkedListNoTail*, en la clase *Node* se incorpora una referencia adicional denominada *prev*, la cual apunta al nodo anterior y permite recorrer la lista en ambos sentidos.

```

private static class Node {
    int data;
    Node next;
    Node prev;
    Node(int data) {
        this.data = data;
    }
}

```

Además, se definen dos atributos principales: *head*, que mantiene la referencia al primer nodo de la estructura, y *size*, que almacena la cantidad actual de elementos de la lista, permitiendo conocer su tamaño sin necesidad de recorrer la lista completamente.

```

private Node head;
private int size;

```

Métodos

- *PushFront* ($O(1)$): El método *pushFront* inserta un nuevo nodo al inicio de la lista doblemente enlazada.

```

if (head != null) {
    head.prev = newNode;
}
head = newNode;
size++;

```

- *PushBack* ($O(n)$): El método *pushBack* inserta un nuevo nodo al final de la lista doblemente enlazada.

```

while (actual.next != null) {
    actual = actual.next;
}
actual.next = newNode;
newNode.prev = actual;
size++;

```

- *PopFront* ($O(1)$): El método *popFront* elimina y retorna el valor almacenado en el primer nodo de la lista doblemente enlazada.

```

public int popFront() {
    if (head == null) {
        return -1;
    }
    int item = head.data;
    head = head.next;
    if (head != null) {
        head.prev = null;
    }
    size--;
    return item;
}

```

- *PopBack* ($O(n)$): El método *popBack* elimina y retorna el valor almacenado en el último nodo de la lista doblemente enlazada.

```

while (actual.next != null) {
    actual = actual.next;
}
int item = actual.data;
actual.prev.next = null;
size--;

return item;

```

- *Erase* ($O(1)$): El método *erase* elimina el valor *item* en la lista doblemente enlazada cuando lo encuentra.

```

if (head.data == item) {
    head = head.next;
    if (head != null) {
        head.prev = null;
    }
    size--;
    return true;
}

```

- *AddBefore* ($O(1)$): El método *addBefore* añade un nuevo nodo con el valor *newItem* antes del primer nodo que contiene el valor *target*.

```

if (head.data == target) {
    pushFront(newItem);
    return;
}

```

- *AddAfter* ($O(1)$): El método *addAfter* añade un nuevo nodo con el valor *newItem* después del primer nodo que contiene el valor *target*.

```

public void addAfter(int target , int newItem) {
    Node actual = head;
    while (actual != null) {
        if (actual.data == target) {
            Node newNode = new Node(newItem);
            newNode.next = actual.next;
            newNode.prev = actual;
            actual.next = newNode;
            if (newNode.next != null) {
                newNode.next.prev = newNode;
            }
            size++;
            return;
        }
        actual = actual.next;
    }
}

```

- *Back* ($O(n)$): El método *back* devuelve el valor del último nodo de la lista.

```

public int back() {
    if (head == null) {
        return -1;
    }
    Node actual = head;
    while (actual.next != null)
        actual = actual.next;
    return actual.data;
}

```

Métodos que no cambian *WithTail* y *NoTail* en *SinglyLinkedList* y *DoublyLinkedList*

- *Find* ($O(n)$): El método *find* devuelve el nodo que contiene el valor especificado, o *null* si no se encuentra en la lista.

```

public Node find(int item) {
    Node actual = head;
    while (actual != null) {
        if (actual.data == item) {
            return actual;
            actual = actual.next;
        }
    }
    return null;
}

```

- *IsEmpty* ($O(1)$): El método *isEmpty* devuelve true si la lista está vacía, o false en caso contrario usando el atributo *size*.

```

public boolean isEmpty() {
    return size == 0;
}

```

- *Size* ($O(1)$): El método *size* devuelve el tamaño de la lista usando el atributo *size*.

```

public int size() {

```

```

        return size;
    }

```

- *Front* ($O(1)$): El método *front* devuelve el valor del primer nodo de la lista.

```

public int front() {
    if (head == null) {
        return -1;
    }
    return head.data;
}

```

2.1.5. *DoublyLinkedListWithTail*: En la implementación de la *DoublyLinkedListWithTail*, en la clase *Node* se incorpora una referencia adicional denominada *prev*, la cual apunta al nodo anterior y permite recorrer la lista en ambos sentidos.

```

private static class Node {
    int data;
    Node next;
    Node prev;
    Node(int data) {
        this.data = data;
    }
}

```

Además, se definieron tres atributos principales: *head*, que mantiene la referencia al primer nodo de la estructura, *tail*, que mantiene la referencia al último nodo de la estructura, y *size*, que almacena la cantidad actual de elementos de la lista, permitiendo conocer su tamaño sin necesidad de recorrerla completamente.

```

private Node head;
private Node tail;
private int size;

```

Métodos

- *PushFront* ($O(1)$): El método *pushFront* inserta un nuevo nodo al inicio de la lista.

```

if (head != null) {
    head.prev = newNode;
} else {
    tail = newNode;
}
head = newNode;
size++;

```

- *PushBack* ($O(1)$): El método *pushBack* inserta un nuevo nodo al final de la lista doblemente enlazada con cola.

```

if (tail != null) {
    tail.next = newNode;
} else {
    head = newNode;
}
tail = newNode;
size++;

```

- *PopFront* ($O(1)$): El método *popFront* elimina y retorna el valor almacenado en el primer nodo de la lista doblemente enlazada.

```

if (head == null) {
    tail = null;
} else {
    head.prev = null;
}
size —;
return item;

```

- *PopBack* ($O(1)$): El método *popBack* elimina y retorna el valor almacenado en el último nodo de la lista doblemente enlazada.

```

if (tail == null) {
    head = null;
} else {
    tail.next = null;
}
size —;
return item;

```

- *Erase* ($O(1)$): El método *erase* elimina la primera aparición del valor especificado en la lista doblemente enlazada con cola.

```

while (actual != null) {
    if (actual.data == item) {
        Node prev = actual.prev;
        Node next = actual.next;
        if (prev != null) {
            prev.next = next;
        }
        if (next != null) {
            next.prev = prev;
        } else {
            tail = prev;
        }
        size —;
        return true;
    }
    actual = actual.next;
}

```

- *AddBefore* ($O(1)$): El método *addBefore* añade un nuevo nodo con el valor *newItem* antes del primer nodo que contiene el valor *target*.

```

if (actual.data == target) {
    Node newNode = new Node(newItem);
    Node prev = actual.prev;
    newNode.prev = prev;
    newNode.next = actual;
    actual.prev = newNode;
    if (prev != null) {
        prev.next = newNode;
    }
    size ++;
    return;
}

```



```

    }
    actual = actual.next;

```

- *AddAfter* ($O(1)$): El método *addAfter* inserta un nuevo elemento después de la primera aparición del valor objetivo en la lista doblemente enlazada con cola.

```

    if (actual.data == target) {
        Node newNode = new Node(newItem);
        Node next = actual.next;

        newNode.prev = actual;
        newNode.next = next;
        actual.next = newNode;
    }

```

- *Back* ($O(1)$): El método *back* devuelve el valor del último nodo de la lista.

```

    public int back() {
        if (tail == null) {
            return -1;
        }
        return tail.data;
    }

```

Métodos que no cambian *WithTail* y *NoTail* en *SinglyLinkedList* y *DoublyLinkedList*

- *Find* ($O(n)$): El método *find* devuelve el nodo que contiene el valor especificado, o *null* si no se encuentra en la lista.

```

    public Node find(int item) {
        Node actual = head;
        while (actual != null) {
            if (actual.data == item) {
                return actual;
            }
            actual = actual.next;
        }
        return null;
    }

```

- *IsEmpty* ($O(1)$): El método *isEmpty* devuelve true si la lista está vacía, o false en caso contrario usando el atributo *size*.

```

    public boolean isEmpty() {
        return size == 0;
    }

```

- *Size*: El método *size* devuelve el tamaño de la lista usando el atributo *size*.

```

    public int size() {
        return size;
    }

```

- *Front* ($O(1)$): El método *front* devuelve el valor del primer nodo de la lista.

```

    public int front() {
        if (head == null) {

```

```

        return -1;
    }
    return head.data;
}

```

2.2. Implementación de *MyStack<T>*

2.2.1. *Estructura*:: La implementación de *MyStack<T>* se realizó mediante un arreglo dinámico genérico. Para ello se utilizaron los atributos *data*, que almacena los elementos de la pila; *size*, que registra la cantidad actual de elementos; y *capacity*, que representa la capacidad máxima del arreglo. Cuando el arreglo alcanza su capacidad, se crea uno nuevo con el doble de tamaño y se copian los elementos existentes, permitiendo que la pila crezca dinámicamente.

```

private T[] data;
private int size;
private int capacity;

```

2.2.2. *Redimensionamiento*:: Cuando el arreglo alcanza su capacidad máxima, se crea un nuevo arreglo con el doble de capacidad y se copian todos los elementos.

```

private void resize() {
    capacity *= 2;
    T[] newData = (T[]) new Object[capacity];

    for (int i = 0; i < size; i++) {
        newData[i] = data[i];
    }
    data = newData;
}

```

Métodos

- *Push (T x)*: El método *push(T x)* inserta un nuevo elemento en la cima de la pila.

```

public void push(T x) {
    if (size == capacity) {
        resize();
    }
    data[size++] = x;
}

```

- *Pop*: El método *pop* elimina y retorna el elemento ubicado en la cima de la pila.

```

public T pop() {
    if (isEmpty()) {
        return null;
    }
    T item = data[--size];
    data[size] = null;
    return item;
}

```

- *Peek*: El método *peek* retorna el elemento ubicado en la cima de la pila sin eliminarlo.

```

public T peek() {
    if (isEmpty()) {

```

```

        return null;
    }
    return data[size - 1];
}

```

- *IsEmpty*: El método *isEmpty* verifica si la pila se encuentra vacía.

```

public boolean isEmpty() {
    return size == 0;
}

```

- *Size*: El método *size* retorna la cantidad actual de elementos almacenados en la pila.

```

public int size() {
    return size;
}

```

- *Delete*: El método *delete* elimina la primera aparición del elemento especificado dentro de la pila.

```

public void delete(T n) {
    for (int i = 0; i < size; i++) {
        if (data[i].equals(n)) {
            for (int j = i; j < size - 1; j++) {
                data[j] = data[j + 1];
            }
            data[--size] = null;
            return;
        }
    }
}

```

2.3. Implementación de *MyQueue<T>*

2.3.1. *Estructura*:: La implementación de *MyQueue<T>* se realizó mediante un arreglo dinámico circular. Para ello se utilizaron los atributos *data*, que almacena los elementos de la cola; *front*, que indica la posición del primer elemento; *rear*, que señala la siguiente posición disponible para inserción; *size*, que registra la cantidad actual de elementos; y *capacity*, que representa la capacidad máxima del arreglo.

```

private T[] data;
private int front;
private int rear;
private int size;
private int capacity;

```

2.3.2. *Redimensionamiento*:: Cuando la cola alcanza su capacidad, se ejecuta el método *resize()*, que crea un nuevo arreglo de mayor tamaño y reorganiza los elementos para mantener el orden de la estructura. La cola sigue el principio FIFO (First In, First Out), por lo que el primer elemento en ingresar es el primero en ser eliminado.

```

private void resize() {
    capacity *= 2;
    T[] newData = (T[]) new Object[capacity];

    for (int i = 0; i < size; i++) {

```

```

        newData[i] = data[(front + i) % data.length];
    }

    data = newData;
    front = 0;
    rear = size;
}

```

Métodos

- *Enqueue*: El método *enqueue* inserta un nuevo elemento al final de la cola.

```

public void enqueue(T x) {
    if (size == capacity) {
        resize();
    }
    data[rear] = x;
    rear = (rear + 1) % capacity;
    size++;
}

```

- *Peek*: El método *peek* retorna el elemento ubicado al frente de la cola sin eliminarlo.

```

public T peek() {
    if (isEmpty()) {
        return null;
    }
    return data[front];
}

```

La expresión `data[front]` permite acceder directamente al primer elemento de la cola. Gracias al uso del índice `front`, no es necesario recorrer la estructura para consultar dicho elemento, por lo que la operación se realiza de manera eficiente.

- *IsEmpty*: El método *isEmpty* verifica si la cola se encuentra vacía. Para ello, compara el valor del atributo *size* con cero. Si no existen elementos almacenados, retorna *true*; en caso contrario, retorna *false*.

```

public boolean isEmpty() {
    return size == 0;
}

```

- *Size*: El método *size* retorna la cantidad actual de elementos almacenados en la cola. Para ello, devuelve directamente el valor del atributo *size*, el cual se actualiza cada vez que se realiza una operación de inserción o eliminación.

```

public int size() {
    return size;
}

```

- *Delete*: El método *delete* elimina la primera aparición del elemento especificado dentro de la cola.

```

public void delete(T n) {
    int index = front;

    for (int i = 0; i < size; i++) {
        if (data[index].equals(n)) {

```

```

    while (index != rear) {
        int next = (index + 1) % capacity;
        data[index] = data[next];
        index = next;
    }
    rear = (rear - 1 + capacity) % capacity;
    data[rear] = null;
    size--;
    return;
}
index = (index + 1) % capacity;
}

```

3. ANÁLISIS DE COMPLEJIDAD

3.1. Hipótesis sobre la eficiencia de cada método

Antes de realizar el análisis de complejidad, se plantean las siguientes hipótesis:

Doubly & Singly LinkedList

1. Las operaciones al inicio de la estructura tendrán una alta eficiencia, ya que únicamente requieren modificar algunas referencias entre nodos.
2. Las implementaciones que incorporan una referencia al último nodo *tail* presentarán un mejor desempeño en las operaciones realizadas al final de la estructura sin referencia, ya que evita recorrer toda la lista.
3. Operaciones *específicas* de búsqueda, eliminación o inserción serán menos eficientes se tenga o no el nodo *tail*, debido a la necesidad de recorrer toda la lista hasta encontrar el elemento.

DinamicArrays

1. En las implementaciones basadas en arreglos dinámicos, se espera que las operaciones de acceso directo, como *peek* y *size*, sean altamente eficientes.
2. Métodos como *delete* y procesos de redimensionamiento del arreglo requerirán un mayor tiempo de ejecución y espacio en memoria debido al desplazamiento o copia de elementos.
3. La implementación de la cola mediante un arreglo circular permitirá realizar inserciones y eliminaciones de manera eficiente, ya que evitará el desplazamiento de elementos después de cada operación y aprovechará mejor el espacio disponible en memoria.

3.2. Análisis de complejidad

- Comparativa de cada método de la Estructura List simple sin cola.

n	PushFront	PushBack	PopFront	PopBack	Find	Erase	AddBefore	AddAfter
10 ¹	7.40	11.40	6.80	9.70	10.00	8.30	14.30	14.50
10 ²	16.30	88.98	2.88	91.13	114.30	50.92	209.20	207.22
10 ³	10.71	794.45	2.77	948.97	1101.08	1048.52	1321.99	1326.82
10 ⁴	10.44	7471.20	2.51	8891.79	10847.41	10786.18	12351.85	17659.93
10 ⁵	3.74	84887.10	3.48	98535.26	143079.43	137726.85	152217.73	217019.42
10 ⁶	5.59	omitido	3.73	omitido	1483747.26	1529281.58	1588209.45	2350956.18
10 ⁷	5.74	omitido	4.51	omitido	21071801.62	21342040.28	22126332.81	33696722.11

Tabla I
SINGLYLINKEDLISTNOTAIL (EN NANOSEGUNDOS)

Los resultados muestran que las operaciones *PushFront* y *PopFront* mantienen tiempos prácticamente constantes independientemente del tamaño de la lista. Esto coincide con la complejidad teórica $O(1)$, ya que únicamente requieren modificar la referencia al nodo *head*.

Por el contrario, las operaciones *PushBack* y *PopBack* presentan un crecimiento aproximadamente lineal. Para tamaños grandes, las mediciones de estas operaciones fueron omitidas debido al tiempo que se requiere para completarlas. Este comportamiento es consistente con una complejidad $O(n)$, dado que es necesario recorrer la lista completa para acceder al último nodo o a su predecesor.

Las operaciones *Find*, *Erase*, *AddBefore* y *AddAfter* también muestran un incremento proporcional al tamaño de la estructura. Los tiempos pasan de decenas de nanosegundos para 10^1 elementos a varios millones de nanosegundos para 10^7 elementos, confirmando la complejidad lineal.

■ Comparativa de cada método de la Estructura List simple con cola.

n	PushFront	PushBack	PopFront	PopBack	Find	Erase	AddBefore	AddAfter
10^1	6.50	6.90	6.20	7.40	9.90	8.90	14.80	18.30
10^2	16.17	15.93	3.32	75.58	114.34	44.37	207.65	206.57
10^3	10.55	11.53	2.65	789.84	1101.08	906.22	1351.36	1324.22
10^4	21.81	22.76	2.42	7857.62	11763.88	9897.37	13189.42	19076.74
10^5	3.37	3.87	4.04	85038.57	126488.49	128109.19	149072.89	223538.10
10^6	6.99	7.46	3.89	omitido	1494265.08	1134515.36	1298702.79	1869553.68
10^7	6.12	7.09	4.34	omitido	20456858.75	19611820.32	21583140.09	35337737.75

Tabla II
SINGLYLINKEDLISTWITHTAIL (EN NANOSEGUNDOS)

La incorporación de una referencia al último nodo produce una mejora significativa en la operación *PushBack*. Los resultados experimentales muestran que su tiempo permanece prácticamente constante incluso para tamaños muy grandes, validando la complejidad teórica $O(1)$.

Sin embargo, la operación *PopBack* continúa requiriendo la búsqueda del penúltimo nodo, por lo que mantiene una complejidad $O(n)$. Este comportamiento se evidencia en el rápido crecimiento de los tiempos de ejecución y en la omisión de algunas mediciones para tamaños muy grandes.

Las operaciones de búsqueda e inserción relativa (*Find*, *Erase*, *AddBefore* y *AddAfter*) conservan una tendencia lineal, ya que todas requieren localizar previamente una posición dentro de la lista.

■ Comparativa de cada método de la Estructura List doble sin cola.

n	PushFront	PushBack	PopFront	PopBack	Find	Erase	AddBefore	AddAfter
10^1	6.60	8.40	7.00	11.50	13.20	12.00	17.10	17.30
10^2	17.38	87.92	2.97	83.86	114.86	49.98	210.37	209.30
10^3	11.72	817.36	2.75	788.82	1037.03	988.68	1334.97	1256.67
10^4	13.36	6153.28	2.26	6170.06	10897.10	11050.24	17708.79	17771.18
10^5	4.45	93615.51	4.37	84813.72	142996.49	142808.46	227784.03	218040.17
10^6	7.31	omitido	4.79	omitido	1230634.11	1237524.41	2078289.15	2167811.89
10^7	6.90	omitido	4.72	omitido	20363360.13	21445497.61	34105352.12	34451650.08

Tabla III
DOUBLYLINKEDLISTNOTAIL (EN NANOSEGUNDOS)

Aunque cada nodo almacena referencias tanto al nodo siguiente como al anterior, la ausencia de una referencia al último nodo obliga a recorrer la lista para acceder al final de la estructura. Por esta razón, las operaciones *PushBack* y *PopBack* continúan mostrando complejidad $O(n)$ y presentan un crecimiento similar al observado en la lista simplemente enlazada sin *tail*.

Las operaciones realizadas sobre la cabeza de la lista mantienen complejidad constante $O(1)$, mientras que las operaciones que requieren localizar nodos específicos conservan comportamiento lineal.

- Comparativa de cada método de la Estructura List doble con cola.

n	PushFront	PushBack	PopFront	PopBack	Find	Erase	AddBefore	AddAfter
10^1	7.10	7.00	6.90	7.00	10.00	12.00	15.00	15.30
10^2	15.71	15.98	3.05	2.97	114.19	47.92	208.62	206.78
10^3	12.00	11.15	2.56	2.57	1038.57	993.98	1274.94	1248.43
10^4	23.95	24.00	2.43	2.43	10916.14	10956.98	17696.73	17907.49
10^5	4.53	4.35	4.41	4.86	136181.91	127301.46	199141.62	202225.17
10^6	7.56	8.26	4.59	4.92	1305027.79	1306301.71	1907672.72	2313410.84
10^7	7.25	7.34	4.77	6.87	20479892.98	20586141.79	34877412.93	34665977.60

Tabla IV
DOUBLYLINKEDLISTWITHTAIL (EN NANOSEGUNDOS)

Esta implementación presenta el mejor desempeño entre todas las listas enlazadas que analizaron. Gracias a la combinación de enlaces dobles y referencias tanto al primer como al último nodo, las operaciones PushFront, PushBack, PopFront y PopBack se ejecutan en tiempo constante $O(1)$.

Los resultados experimentales muestran que los tiempos de estas operaciones permanecen prácticamente invariables al aumentar el tamaño de la estructura, confirmando la predicción teórica.

No obstante, las operaciones Find, Erase, AddBefore y AddAfter continúan dependiendo de la localización de un nodo dentro de la lista, por lo que mantienen complejidad $O(n)$.

- Comparativa de cada método de la Estructura MyStack.

n	push	pop
10^1	10.50	7.00
10^2	5.94	3.61
10^3	6.38	3.24
10^4	7.73	3.27
10^5	4.19	3.67
10^6	4.57	3.80
10^7	6.37	4.29

Tabla V
STACK DINÁMICO (EN NANOSEGUNDOS)

La implementación de la pila mediante un arreglo dinámico presenta un comportamiento estable para las operaciones push y pop. Los tiempos medidos permanecen prácticamente constantes incluso al incrementar el tamaño de la estructura desde 10^1 hasta 10^7 elementos.

Estos resultados concuerdan con la complejidad teórica de $O(1)$ para push y $O(1)$ para pop.

- Comparativa de cada método de la Estructura MyQueue.

n	enqueue	dequeue
10^1	11.50	7.00
10^2	5.79	9.55
10^3	6.36	6.24
10^4	7.70	6.64
10^5	3.69	4.64
10^6	4.49	5.25
10^7	6.85	7.61

Tabla VI
QUEUE DINÁMICA (EN NANOSEGUNDOS)

La cola implementada mediante un arreglo dinámico también presenta tiempos prácticamente constantes para las operaciones enqueue y dequeue. Esto indica que la estructura evita desplazamientos masivos de elementos y utiliza una estrategia eficiente, como un buffer circular.

Los resultados experimentales respaldan la complejidad amortizada $O(1)$ para ambas operaciones principales.

3.3. Comparativa de métodos equivalentes.

Para cada operación se seleccionó la implementación de lista más eficiente posible, con el fin de realizar una comparación justa frente a las estructuras basadas en arreglos.

■ PushFront (List) vs. Push (MyStack)

La operación equivalente a push en una pila es PushFront sobre una lista.

Para esta comparación se eligió DoublyLinkedListWithTail, aunque cualquiera de las cuatro implementaciones ofrece complejidad $O(1)$ para insertar al inicio de la lista.

n	PushFront (DLL With Tail)	Push (MyStack)
10^1	7.10	10.50
10^2	15.71	5.94
10^3	12.00	6.38
10^4	23.95	7.73
10^5	–	4.19
10^6	–	4.57
10^7	–	6.37

Tabla VII
PUSHFRONT (LIST) VS PUSH (MYSTACK) [EN NANOSEGUNDOS]

Aunque ambas operaciones son $O(1)$, la pila implementada mediante arreglo dinámico tiene menores tiempos promedio debido a la ausencia de asignaciones dinámicas de nodos.

■ PopFront (List) vs. Pop (MyStack)

La operación equivalente a pop en una pila es PopFront en una lista.

Se eligió nuevamente DoublyLinkedListWithTail, ya que ofrece complejidad constante.

n	PopFront (DLL With Tail)	Pop (MyStack)
10^1	6.90	7.00
10^2	3.05	3.61
10^3	2.56	3.24
10^4	2.43	3.27
10^5	–	3.67
10^6	–	3.80
10^7	–	4.29

Tabla VIII
POPFront (LIST) VS POP (MYSTACK) [NS]

Las dos estructuras mantienen tiempos constantes. En tamaños pequeños las diferencias son mínimas y ambas validan experimentalmente la complejidad $O(1)$.

■ PushBack (List) vs. Enqueue (MyQueue)

La operación equivalente a enqueue en una cola es PushBack en una lista.

Para esta comparación se eligió DoublyLinkedListWithTail, ya que junto con SinglyLinkedListWithTail es la única implementación que permite inserción al final en tiempo constante.

n	PushBack (DLL With Tail)	Enqueue (MyQueue)
10^1	7.00	11.50
10^2	15.98	5.79
10^3	11.15	6.36
10^4	24.00	7.70
10^5	–	3.69
10^6	–	4.49
10^7	–	6.85

Tabla IX
PUSHBACK (LIST) vs ENQUEUE (MYQUEUE) [NS]

La presencia del puntero tail permite que la inserción al final de la lista sea $O(1)$. Sin embargo, la cola circular basada en arreglos sigue mostrando menores tiempos absolutos debido al acceso contiguo a memoria.

■ PopFront (List) vs Dequeue (MyQueue)

La operación equivalente a dequeue en una cola es PopFront en una lista.

Se seleccionó DoublyLinkedListWithTail, que ejecuta la operación en tiempo constante.

n	PopFront (DLL With Tail)	Dequeue (MyQueue)
10^1	6.90	7.00
10^2	3.05	9.55
10^3	2.56	6.24
10^4	2.43	6.64
10^5	–	4.64
10^6	–	5.25
10^7	–	7.61

Tabla X
POPFONT (LIST) vs DEQUEUE (MYQUEUE) [NS]

Ambas operaciones son constantes. La eliminación en la lista enlazada únicamente requiere actualizar referencias, mientras que en la cola circular se incrementa el índice frontal. Los tiempos medidos confirman que ninguna depende del tamaño de la estructura.

3.4. Complejidad teórica en notación Big-O

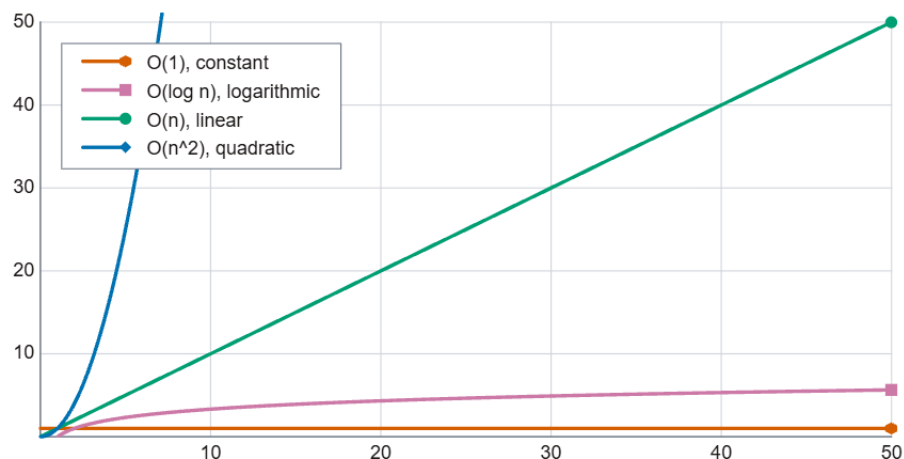


Figura 1. Comparación visual del crecimiento de distintas complejidades temporales [1].

Las Tablas XI y XII resumen la complejidad teórica de cada operación según la implementación. Las celdas en negrita marcan dónde la referencia *tail* cambia el orden de complejidad

respecto a la lista sin cola: *pushBack* y *back* pasan de $O(n)$ a $O(1)$ tanto en la lista simple como en la doble, mientras que *popBack* solo llega a $O(1)$ en la lista doble con cola, pues en la simple sigue siendo necesario recorrer hasta el penúltimo nodo. Las operaciones sobre la cabeza (*pushFront*, *popFront* y *front*) y las consultas (*size* e *isEmpty*) son $O(1)$ en todos los casos. Por su parte, *find*, *erase*, *addBefore* y *addAfter* son $O(n)$ en las cuatro listas, ya que buscan por valor y deben recorrer la estructura; la inserción o el borrado en sí cuesta $O(1)$ únicamente cuando ya se dispone del nodo.

Operación	Simple sin tail	Simple con tail	Doble sin tail	Doble con tail
<i>pushFront</i>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<i>pushBack</i>	$O(n)$	$O(1)$	$O(n)$	$O(1)$
<i>popFront</i>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<i>popBack</i>	$O(n)$	$O(n)$	$O(n)$	$O(1)$
<i>front</i>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<i>back</i>	$O(n)$	$O(1)$	$O(n)$	$O(1)$
<i>find</i>	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<i>erase</i>	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<i>addBefore</i>	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<i>addAfter</i>	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<i>isEmpty</i> / <i>size</i>	$O(1)$	$O(1)$	$O(1)$	$O(1)$

Tabla XI
COMPLEJIDAD TEMPORAL TEÓRICA DE LAS LISTAS ENLAZADAS

Estructura	Operaciones principales	peek, size, isEmpty	delete
MyStack<T>	push, pop: $O(1)$ amortizado	$O(1)$	$O(n)$
MyQueue<T>	enqueue, dequeue: $O(1)$ amortizado	$O(1)$	$O(n)$

Tabla XII
COMPLEJIDAD TEMPORAL TEÓRICA DE MyStack<T> y MyQueue<T>

En *MyStack*< *T* > y *MyQueue*< *T* >, las operaciones principales son $O(1)$ amortizado: el redimensionamiento cuesta $O(n)$, pero ocurre solo cuando se duplica la capacidad. *peek*, *size* e *isEmpty* son $O(1)$, y *delete* es $O(n)$ por la búsqueda y el desplazamiento de elementos. Los tiempos medidos en las Tablas I a VI son consistentes con estas complejidades.

3.5. Complejidad empírica

1. *Operaciones $O(1)$* : Coinciden con la teoría, al multiplicar *n* por 10, el tiempo total se multiplica por unos 10. El arreglo dinámico confirma el costo amortizado. Aunque duplica su capacidad y copia de vez en cuando, *push* cuesta unos 7 ns, del mismo orden que insertar en una lista enlazada.
2. *Operaciones $O(n)$ repetidas *n* veces*: Es la confirmación más clara, 10 veces más datos cuesta unas 100 veces más tiempo. Extrapolando, *pushBack* sin cola tardaría unos 15 minutos con 10^6 y más de un día con 10^7 . Por eso esas 10 filas están marcadas como omitidas. El puntero a la cola convierte *pushBack* en $O(1)$: con 10^5 tarda 0,47 ms frente a 9.347 ms sin él. Para *popBack* la cola no basta, porque en la lista simple con cola sigue haciendo falta recorrer hasta el penúltimo nodo (9.379 ms con 10^5). Solo la lista doble con cola lo resuelve en $O(1)$ (0,54 ms).
3. *find*, *erase*, *addBefore*, *addAfter*. Son $O(n)$ en las cuatro listas, con o sin cola, porque hay que recorrer hasta el valor ($n/2$ nodos en promedio). El exponente medido es $\approx 1,05$ –1,1, y sube a 1,1–1,3 entre 10^6 y 10^7 . Aproximando el recorrido en $n/2$ nodos, el costo por nodo

pasa de $\approx 2,9ns$ (10^5 – 10^6) a $\approx 5,3ns$ (10^7). Mi hipótesis es que la lista deja de caber en caché, pero no medí contadores de hardware. Big-O no captura esa constante. Además, las variantes de igual orden difieren hasta $\sim 1,5\times$ (por ejemplo, *addBefore* con 10^5 tarda $15ms$ en las simples y $23ms$ en las dobles).

4. *delete(T x)* en pila y cola: En teoría es $O(n)$, buscar el valor ($n/2$ elementos en promedio) más desplazar los elementos posteriores con arraycopy. Con 10^7 elementos, cada *delete* tarda $\approx 12,2ms$ en MyStack y $\approx 8,5ms$ en MyQueue. Con 10^5 son $\approx 68\mu s$ y $\approx 41\mu s$. Tienen el mismo orden de crecimiento, aunque la pila resulta $\sim 1,4\times$ más lenta.

3.6. Justificación de medición en nanosegundos

1. Las operaciones $O(1)$ son demasiado rápidas para los milisegundos. Un *pushFront* o un *pop* dura decenas o cientos de nanosegundos. Con un método que redondee a 0 ms, no se podría distinguir una operación de otra ni ver si el costo crece con n.
2. Permite comparar métodos de órdenes de magnitud muy distintos. En la misma medición conviven operaciones $O(1)$ (nanosegundos) y $O(n)$ (hasta milisegundos con n grande). Al medir en la unidad más fina, ninguna pierde precisión, y solo se cambia de escala para graficar.

4. VISUALIZACIÓN Y GRAFICACIÓN DE RESULTADOS

4.1. Singly Linked List - no Tail

La gráfica muestra una clara separación entre las operaciones realizadas al inicio de la lista y aquellas que involucran posiciones cercanas al final. Mientras las primeras permanecen prácticamente horizontales, las demás presentan una pendiente creciente que se vuelve más pronunciada a medida que aumenta el tamaño de la estructura. Las curvas correspondientes a *Find*, *Erase*, *AddBefore* y *AddAfter* terminan concentrándose en la zona superior del gráfico para tamaños grandes.

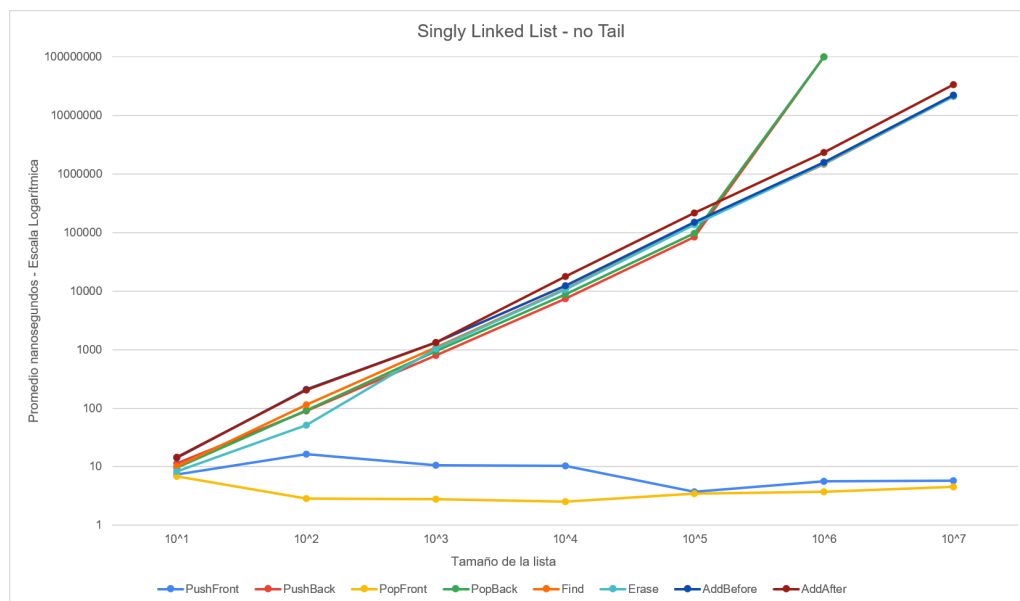


Figura 2. Singly Linked List - no Tail

4.2. Singly Linked List - with Tail

En esta implementación destaca el comportamiento de *PushBack*, cuya curva se mantiene cercana a las operaciones constantes durante todo el experimento. Además, se logra apreciar una diferencia importante respecto a *PopBack*, que continúa creciendo con el tamaño de la lista. Esto genera dos grupos claramente diferenciados de operaciones dentro de la misma estructura.

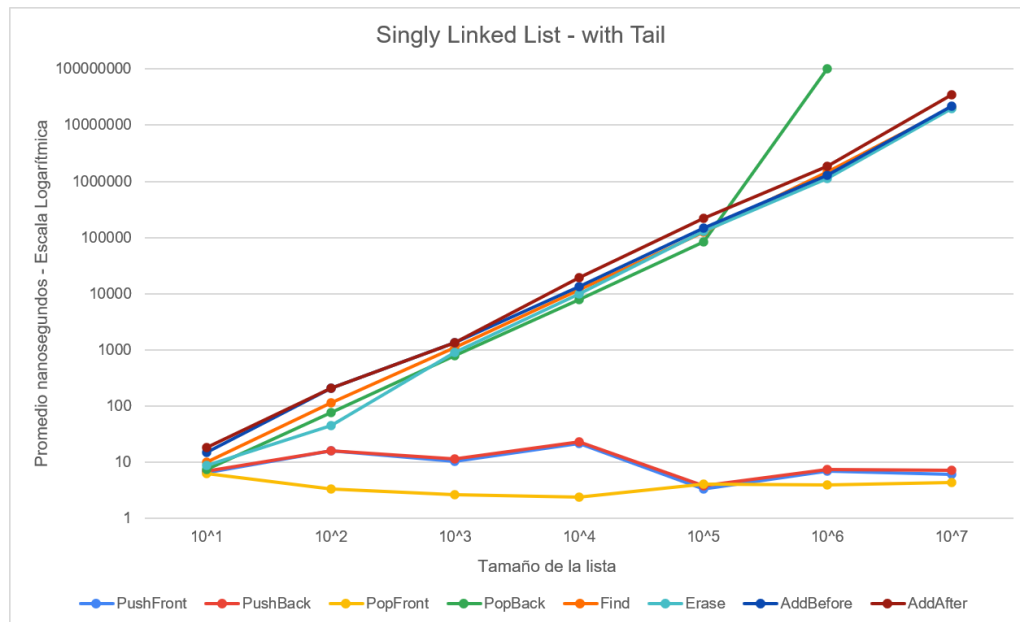


Figura 3. Singly Linked List - with Tail

4.3. Doubly Linked List - no Tail

Las curvas observadas presentan un patrón similar al de la lista simplemente enlazada sin tail. Las operaciones asociadas al final de la estructura muestran un crecimiento progresivo, mientras que las operaciones sobre la cabeza permanecen estables. La escala logarítmica permite apreciar simultáneamente ambos comportamientos a pesar de la gran diferencia entre sus magnitudes.

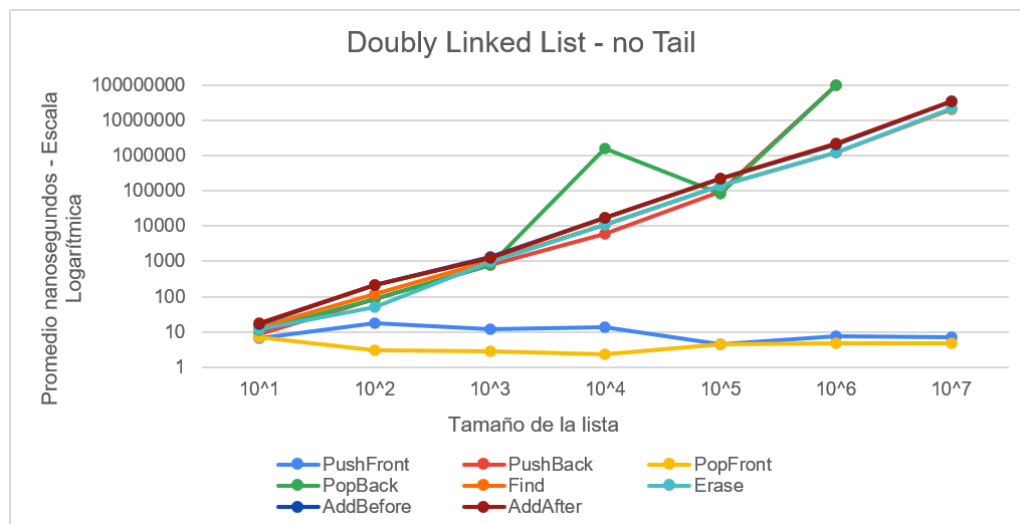


Figura 4. Doubly Linked List - no Tail

4.4. Doubly Linked List - with Tail

La mayoría de las curvas permanecen agrupadas en la parte inferior de la gráfica, indicando tiempos de ejecución muy similares para las operaciones sobre los extremos de la lista. Las únicas curvas que muestran un crecimiento apreciable son las relacionadas con búsqueda e inserción relativa, lo que produce una visualización más compacta que en las demás implementaciones de listas.

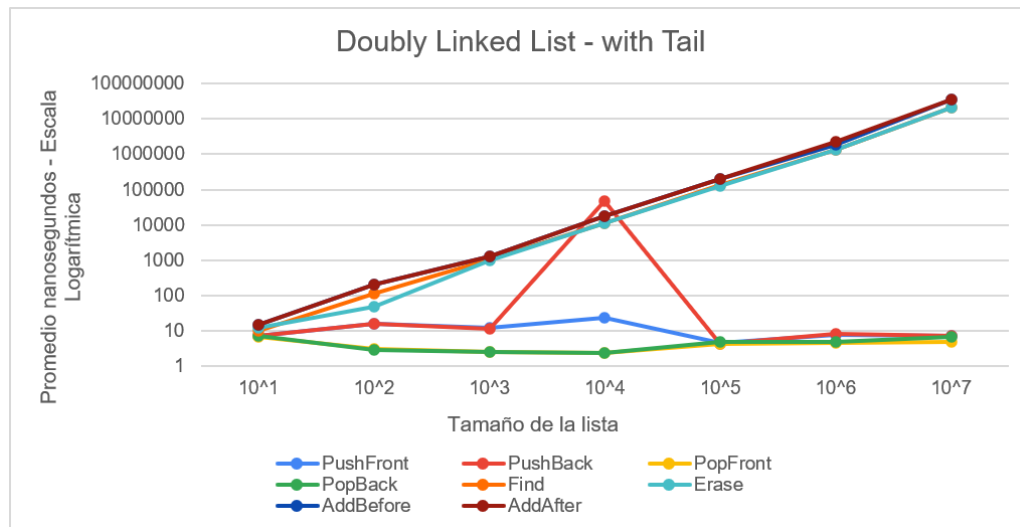


Figura 5. Doubly Linked List - with Tail

4.5. $MyStack < T > \& MyQueue < T >$

Las dos curvas de Stack representadas se mantienen prácticamente superpuestas a lo largo de todo el rango de tamaños evaluados. La ausencia de incrementos significativos confirma la estabilidad del rendimiento de la estructura, incluso cuando el número de elementos aumenta en varios órdenes de magnitud.

Por otro lado, las curvas de Queue evidencian una variación mínima entre las operaciones de inserción y eliminación. Ambas curvas siguen trayectorias muy similares y permanecen cercanas durante todo el experimento, mostrando un comportamiento uniforme para tamaños pequeños y grandes.

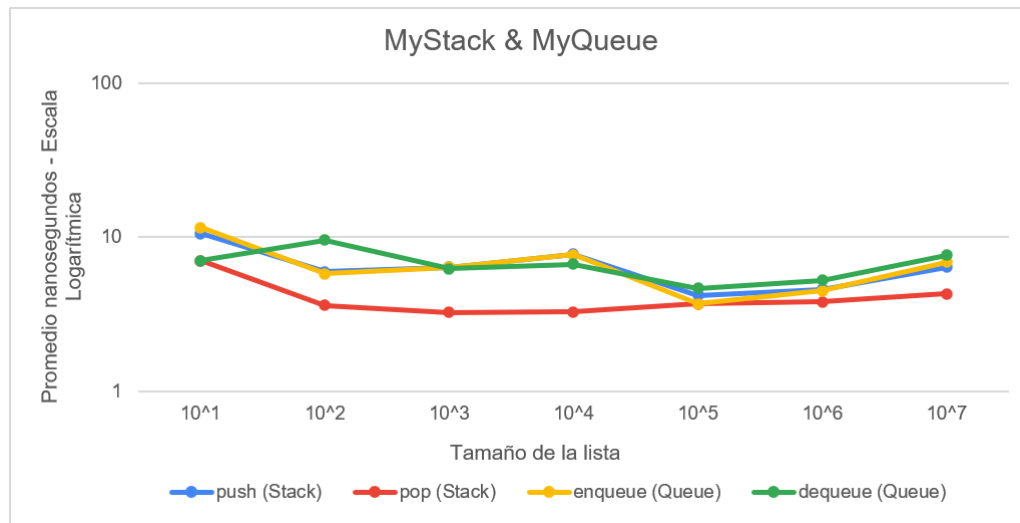


Figura 6. $MyStack < T > \& MyQueue < T >$

5. CONCLUSIONES

5.1. Conveniencia

Primeramente, se evidencia que insertar y eliminar en los extremos es barato en todas las estructuras: (unos 5–7 ns por operación), con un crecimiento claramente lineal. Entonces no hay diferencia que valga la pena.

Donde sí hay diferencia es al insertar al final. El puntero a la cola hace que sea lineal $O(1)$: con 10^5 elementos, pasa de 9,3 s a 0,47 ms. Pero tail no sirve para todo: eliminar al final solo es $O(1)$ en la lista doble con cola (0,54 ms con 10^5). En las otras tres estructuras toca recorrer hasta el penúltimo nodo, y eso se traduce en 9,4–11 s. En la lista simple, tener cola no alcanza.

Con eso en mente, la elección sería de esta forma:

- Pila, cuando es lista simple sin cola o arreglo dinámico.
- Cola FIFO, cuando es lista simple con cola o arreglo dinámico.
- Deque $\rightarrow$ lista doble con cola.

Si se necesita buscar o modificar por valor, todas las implementaciones son $O(n)$: entre 25–34 ms por llamada en listas con 10^7 elementos, y 8,5–12 ms en pila y cola con arreglo. La cola no ayuda aquí. Si ese tipo de operaciones es frecuente, ninguna de las opciones es conveniente.

5.2. Uso de pilas y colas en aplicaciones reales

- Las pilas (LIFO) aparecen en "Deshacer"(Ctrl + Z) en los editores de texto; en el botón "Atrás" de los navegadores; el sistema operativo usa una pila de ejecución para saber qué función se está ejecutando y a cuál debe regresar cuando termine.
- Las colas (FIFO) son una una fila para un banco, aparecen en la gestión de impresión; reproducción de música y videos; en peticiones o eventos.

5.3. Ventajas y desventajas de arreglos dinámicos vs. listas enlazadas

5.3.1. Ventajas DinamicArrays:

- Sus elementos son contiguos en memoria, lo que mejora el uso de la caché.
- La sobrecarga por elemento es menor y el acceso por índice es $O(1)$.
- *Delete* en MyQueue $\approx 8,5$ ms con 10^7 elementos) resulta unas tres veces más rápido que *erase* en las listas (25–28 ms), incluso a pesar de que además desplaza elementos.

5.3.2. Desventajas DinamicArrays:

- Cuando se llena, copia todo su contenido, lo que tiene un costo $O(n)$ y puede quedar capacidad sin utilizar.
- Insertar o eliminar en el medio o al inicio obliga a desplazar elementos.

5.3.3. Ventajas LinkedList:

- No requiere redimensionamiento, cada operación tiene un costo estable, y con cola o doble enlace ofrece $O(1)$ en los extremos.
- Si se dispone del nodo, permite insertar o borrar en $O(1)$.

5.3.4. Desventajas LinkedList:

- El acceso por posición y la búsqueda son $O(n)$.
- Existe un objeto por nodo, lo que implica más memoria y mayor carga para el recolector de basura. Los nodos quedan dispersos en memoria.

6. REPOSITORIO EN GITHUB

[Repositorio GitHub del proyecto](#)

7. REFERENCIAS

REFERENCIAS

- [1] S. Rose. (2025) Big o. Accessed: 2026-09-28. [Online]. Available: <https://samwho.dev/big-o>